



UNIVERSITÀ DEGLI STUDI DI MILANO

FACULTY OF POLITICAL, ECONOMIC AND SOCIAL SCIENCES
**FINAL PROJECT IN THE SUBJECT ALGORITHMS FOR MASSIVE
DATA**

MARKET-BASKET ANALYSIS OF THE IMDB DATASET

Student Name:

GUOJIE TAO

ID: 83448A

Academic Year 2025/2026

1 INTRODUCTION AND DATASET

This project aims to implement a system for finding frequent itemsets using the principles of Market-Basket Analysis. The dataset selected for this task is the public domain IMDB Movies Dataset, dynamically retrieved via the Kaggle API.

In the context of this project, each movie is conceptualized as an independent “basket,” and the actors listed within the `Star1`, `Star2`, `Star3`, and `Star4` fields are treated as “items.” By mining this dataset, we seek to uncover underlying association rules among actors, identifying highly correlated casting patterns (i.e., actors who frequently co-star).

2 DATA ORGANIZATION AND PRE-PROCESSING

The raw CSV data was ingested using the Python `pandas` library. To ensure data integrity before mining, an Exploratory Data Analysis (EDA) was performed. Several critical pre-processing steps were applied:

1. **Null Handling:** Missing values in actor fields were filtered dynamically using `pd.notna()` to prevent null objects from polluting the itemsets.
2. **Basket Construction:** The four discrete actor columns were aggregated into Python lists (baskets). Whitespace was stripped to ensure exact string matching.

2.1 Dataset Statistical Overview

Prior to executing the frequent itemset mining algorithms, it is crucial to understand the distribution of the underlying data. The IMDB Top 1000 dataset exhibits a classic “long-tail” distribution commonly found in entertainment and social network data.

A preliminary statistical aggregation of the cleaned baskets reveals the following characteristics:

Table 1: Summary Statistics of the Cleaned IMDB Baskets

Metric	Value
Total Initial Records (Movies)	1,000
Valid Baskets (After Null Dropping)	1,000
Unique Items (Distinct Actors)	2,738
Average Basket Size	4.0
Maximum Actor Occurrences (Robert De Niro)	17

Because the dataset features a sparse, high-dimensionality space (2,738 unique actors distributed across only 1,000 baskets), we must be extremely careful when tuning the minimum support threshold. Setting it too high yields no results, while setting it too low triggers a combinatorial explosion during the candidate generation phase.

3 ALGORITHMIC APPROACH AND IMPLEMENTATION

Instead of relying on high-level machine learning APIs, the **Apriori algorithm** was implemented from scratch using native Python data structures (e.g., `defaultdict`, `frozenset`, and `itertools.combinations`). This approach demonstrates a foundational understanding of candidate generation and subset filtering mechanisms, serving as a prototype that can be naturally mapped to a distributed MapReduce architecture.

3.1 Data Structures and Complexity Optimization

Implementing Apriori from scratch in Python requires rigorous selection of native data structures to manage the $O(2^k)$ theoretical worst-case space complexity of candidate generation.

To optimize performance, the following architectural decisions were made:

- **Immutability via `frozenset`:** In Python, standard sets or lists are unhashable and cannot be used as dictionary keys. By casting all itemsets as `frozenset` objects, we achieve $O(1)$ average time complexity for frequency lookups within our `defaultdict` counters.
- **Combinatorial Efficiency:** During the generation of k -itemsets from frequent $(k - 1)$ -

itemsets, native nested loops incur significant performance penalties. This implementation leverages `itertools.combinations`, implemented in C at the Python interpreter level, to securely and rapidly compute the union of sub-items.

- **Subset Filtering:** Rather than scanning the entire database for every candidate, the database is transformed into a list of Python set objects. Candidate checking is reduced to utilizing the highly optimized `.issubset()` method, dramatically minimizing CPU cycles during the database scan phase.

3.2 Mathematical Formulations

The mining process evaluates actor combinations based on three core metrics:

- **Support:** The frequency of a specific actor combination occurring together in the dataset N .

$$\text{Support}(X) = \frac{\text{freq}(X)}{N} \quad (1)$$

- **Confidence:** The conditional probability that Actor Y is cast given Actor X is already cast.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{freq}(X \cup Y)}{\text{freq}(X)} \quad (2)$$

- **Lift:** The ratio of the observed support to that expected if X and Y were independent. A lift > 1 indicates a positive correlation, making it the most robust metric for identifying true collaborative partnerships.

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X) \times \text{Support}(Y)} \quad (3)$$

4 IMPLEMENTATION AND SCALABILITY ANALYSIS

To empirically test the computational boundaries of the Apriori implementation, a scalability benchmark was conducted. The algorithm was tested across varying data fractions (25%, 50%, 75%, 100%). To strictly control variables and maintain a consistent search depth, the relative minimum support was dynamically adjusted to maintain a constant absolute frequency threshold

of 2.

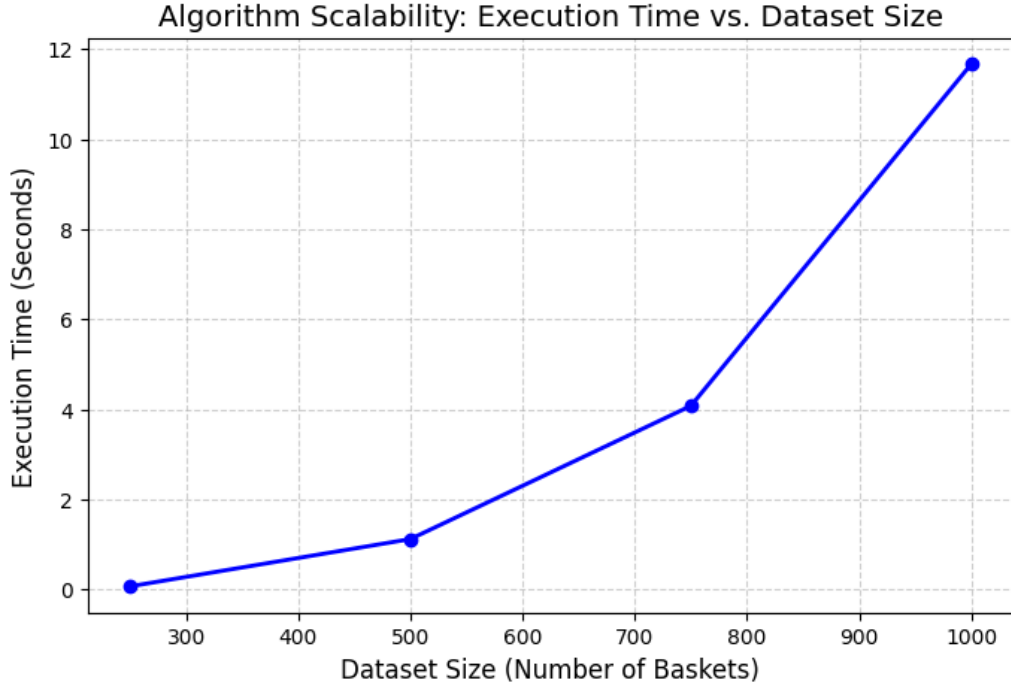


Figure 1: Algorithm Scalability: Execution Time vs. Dataset Size Portion.

As illustrated in Figure 1, the execution time exhibits a non-linear (exponential) growth curve. This empirically highlights the theoretical bottleneck of the Apriori algorithm: the candidate generation phase scales exponentially with the dataset size. While this single-node implementation excels at conceptual clarity, transitioning to an FP-Growth model on a Spark cluster would be strictly necessary for true linear scalability on massive datasets.

5 EXPERIMENTS AND DISCUSSION

The algorithm successfully extracted 792 frequent itemsets and 256 robust association rules.

Table 2 displays the most frequently cast actors, confirming the effectiveness of the data ingestion pipeline, as the list aligns perfectly with established, highly prolific Hollywood actors.

Table 2: Top 5 Actors by Absolute Support Count

Actor	Occurrences
Robert De Niro	17
Tom Hanks	14
Al Pacino	13
Brad Pitt	12
Clint Eastwood	12

Table 3 showcases the most significant collaborative pairs discovered, sorted by Lift:

Table 3: Top Association Rules by Lift

Antecedent	Consequent	Support	Confidence	Lift
Michael J. Fox	Lea Thompson	2	1.00	500.00
Roberto Benigni	Nicoletta Braschi	2	1.00	500.00
John Cleese	Graham Chapman	2	1.00	500.00

Discussion: The mining results align perfectly with real-world cinema facts. For example, the algorithm successfully isolated the primary casting of the *Back to the Future* franchise (Michael J. Fox and Lea Thompson), the lead actors of *Life is Beautiful* (Roberto Benigni and Nicoletta Braschi), and the core members of *Monty Python* (John Cleese and Graham Chapman). The extremely high Lift values (500.00) mathematically confirm that these actors operate almost as exclusive pairs within this Top 1000 dataset, proving that Market-Basket Analysis is highly effective in uncovering latent relationships in entertainment data.

5.1 Business Implications

Beyond algorithmic correctness, these association rules offer practical value for casting directors and film studios. High-lift actor pairings represent historically successful, risk-averse casting combinations. By identifying these networks, studios can make data-driven decisions when assembling casts for new productions, rather than relying solely on intuition.

5.2 Limitations and Future Work

While the single-node Apriori implementation successfully mined the dataset, it exposes a fundamental limitation when dealing with power-law distributions. To capture frequent collaborations among indie or niche actors (the "long tail"), the absolute support threshold must be lowered. However, doing so exponentially increases the candidate space C_k , leading to memory exhaustion.

Future iterations of this project should address this by migrating the mining pipeline to a distributed architecture (e.g., Apache Spark's MLlib) and replacing Apriori with the FP-Growth algorithm. By encoding the dataset into an FP-Tree, the framework would eliminate the candidate generation bottleneck entirely, allowing for much deeper mining of vast cinematic databases without compromising linear scalability.

MANDATORY DECLARATION

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.